

VIBE CODING & AGENTIC DEVELOPMENT

A Progressive Field Guide for MacOS
From Terminal Basics to Fluent Agentic Coding

Tools Covered: Terminal · nvm · Git · Gemini CLI · Antigravity IDE · Google AI Studio

How to Use This Guide

This guide is built in two layers. The first layer is a progressive curriculum — six modules that build on each other. Work through them in order. The second layer is a reference: every command, concept, and pitfall is documented so you can search and return to any section later.

You don't need to memorize everything. The goal is to reach fluency — the point where you understand what is happening, can read errors without panic, and know which tool to reach for.

The six modules:

- Module 1 — The Terminal: Your New Home
- Module 2 — Git: Saving Your Work Like a Pro
- Module 3 — Node, npm, and Your Toolchain
- Module 4 — Gemini CLI: AI in the Terminal
- Module 5 — Antigravity IDE: Agentic Coding in Practice
- Module 6 — Fluency: Projects, Patterns, and Pitfalls

□ **How to read this guide:** Read the concept section first. Then do the exercises. Then use the reference tables when you forget something. Repeat.

✓ **Prerequisites:** You have a Mac, a terminal open, nvm installed, and Gemini CLI set up. That's it. You already did the hard part.

MODULE 1

The Terminal: Your New Home

Stop fearing it. Start commanding it.

1.1 What the Terminal Actually Is

Your Mac has two interfaces. One is the graphical desktop you see — windows, icons, drag and drop. The other is the terminal: a text interface where you type commands and the computer executes them immediately.

The terminal isn't old or inferior. Professionals prefer it because it's faster, scriptable, and far more powerful for development work. When you installed `nvm` earlier, you were doing something a graphical installer could not do as cleanly.

The shell is the program inside the terminal that interprets your commands. On modern Macs, the default shell is `zsh` (Z Shell). When you see a prompt ending in `%`, that's `zsh` waiting for input.

1.2 Anatomy of the Prompt

```
pedroamaral@Pedros-MBP ~ %  
├── The % means zsh is ready for input  
├── ~ means you are in your home directory  
├── Your Mac's hostname  
└── Your username
```

1.3 Navigation: Moving Around Your Files

Your files are organized in a tree of folders. The terminal always has a current location — called the working directory. Every command you run happens relative to that location.

THE ESSENTIAL NAVIGATION COMMANDS

```
pwd          # Print Working Directory - shows where you are right now  
ls           # List files in the current directory  
ls -la       # List ALL files (including hidden ones) with details  
cd Documents # Change Directory - move into the Documents folder  
cd ..        # Go up one level (to the parent folder)  
cd ~         # Go to your home directory from anywhere  
cd -         # Go back to the previous directory (like a Back button)
```

❏ **Pro tip:** Tab completion is your best friend. Type the first few letters of a folder name and press Tab — the shell completes it for you. Press Tab twice to see all options.

1.4 Files and Folders

CREATING AND MANAGING FILES

```
mkdir my-project          # Make a new directory called my-project
mkdir -p a/b/c            # Create nested directories all at once
touch index.html          # Create an empty file called index.html
cp file.txt backup.txt    # Copy a file
mv file.txt folder/       # Move a file into a folder
mv old.txt new.txt        # Rename a file (move = rename in terminal)
rm file.txt               # Delete a file (no trash – gone immediately)
rm -rf folder/            # Delete a folder and everything inside it
```

❏ **Pitfall:** `rm -rf` cannot be undone. There is no Trash. Before running it, double-check your current directory with `pwd` and list files with `ls`.

1.5 Reading Files

VIEWING FILE CONTENTS

```
cat file.txt              # Print the entire file to screen
less file.txt             # Scroll through a file (q to quit)
head -20 file.txt         # Show the first 20 lines
tail -20 file.txt         # Show the last 20 lines
tail -f logfile.log       # Watch a file as it updates in real time
```

1.6 Understanding Paths

Every file on your Mac has an address called a path. There are two kinds:

- Absolute path: starts from the root of the filesystem with `/`. Example: `/Users/pedroamaral/Documents/project`
- Relative path: starts from your current directory. Example: `Documents/project` (when you're already in your home folder)

The shorthand `~` always means your home directory (`/Users/pedroamaral` on your machine). You'll see it constantly.

1.7 Environment Variables

Environment variables are named values the shell stores in memory. They configure how programs behave. You'll encounter them when setting API keys and configuring tools.

WORKING WITH ENVIRONMENT VARIABLES

```
echo $HOME                # Print the HOME variable (your home dir)
echo $PATH                # Print PATH – where the shell looks for programs
```

```
export MY_KEY=abc123      # Set a variable for this terminal session
echo $MY_KEY              # Verify it was set
```

❑ **Pitfall:** Variables set with `export` only last for the current terminal session. When you close the terminal, they're gone. To make them permanent, add them to `~/.zshrc`.

1.8 The `.zshrc` File — Your Shell's Configuration

When `zsh` starts, it reads a file called `.zshrc` in your home directory. This is where permanent settings live: aliases, environment variables, and shell behavior. It's a plain text file you edit directly.

EDITING YOUR `.ZSHRC`

```
nano ~/.zshrc          # Open .zshrc in the nano editor
# Make your changes, then press Ctrl+O to save, Ctrl+X to exit

source ~/.zshrc        # Reload the file without restarting the terminal
```

Useful things to add to your `.zshrc`:

```
# Shortcuts (aliases)
alias ll='ls -la'
alias ..='cd ..'
alias gs='git status'

# Make a permanent API key available
export GEMINI_API_KEY=your_key_here
```

1.9 Processes and Control

When you run a command that doesn't stop on its own (like a web server), it runs as a process. Knowing how to control processes is critical.

PROCESS CONTROL

```
Ctrl+C      # Stop the currently running process
Ctrl+Z      # Pause a process (put it in background)
Ctrl+D      # Send end-of-input / close the terminal
Ctrl+L      # Clear the screen (same as: clear)
Ctrl+A      # Jump to beginning of the line
Ctrl+E      # Jump to end of the line
Ctrl+R      # Search your command history
```

1.10 Pipes and Redirection

The real power of the terminal comes from chaining commands together. The `|` character (pipe) takes the output of one command and feeds it as input to the next.

PIPES AND REDIRECTION

```
ls -la | grep '.js'           # List files, filter to only show .js files
cat log.txt | grep 'error'    # Show only lines containing 'error'
echo 'hello' > file.txt       # Write 'hello' to a file (overwrites)
echo 'world' >> file.txt       # Append 'world' to a file
npm install 2>&1 | tee install.log # Log output to file AND screen
```

1.11 Module 1 Exercises

Work through these in order. Don't skip ahead.

1. Open a terminal. Run `pwd`. Write down exactly where you are.
2. Run `ls -la` in your home directory. Find the hidden files (they start with a dot).
3. Create a folder: `mkdir ~/vibe-projects`
4. Navigate into it: `cd ~/vibe-projects`
5. Create a file: `touch README.md`
6. Open your `.zshrc` with `nano ~/.zshrc` and add the alias `ll='ls -la'`. Save and reload.
7. Run `ll` and confirm it works.
8. Run `echo $PATH` and look at the output. Notice your `nvm` path is in there — that's how the shell finds the right node.

MODULE 2

Git: Saving Your Work Like a Pro

Version control is your safety net. Never code without it.

2.1 Why Git Matters

Git is a version control system. It tracks every change you make to your code, lets you go back to any previous state, and enables you to work on different features simultaneously without breaking anything.

Think of it like a time machine for your project. Every time you 'commit', you create a snapshot. If you break something, you can rewind. If you want to try a risky experiment, you create a 'branch' — a parallel version — and only merge it back if it works.

For agentic coding specifically, Git is non-negotiable. AI agents can make sweeping changes across many files simultaneously. Without Git, one bad run can destroy work you can't recover.

2.2 Core Concepts

- Repository (repo): A folder that Git is tracking. Has a hidden .git subfolder containing all history.
- Commit: A saved snapshot of your project at a point in time. Has a unique ID and a message describing what changed.
- Branch: A parallel version of your project. The default is called main.
- Staging area: A holding area where you prepare changes before committing them.
- Remote: A copy of your repo hosted elsewhere (like GitHub). Lets you back up and collaborate.

2.3 Setting Up Git

FIRST TIME SETUP (DO THIS ONCE)

```
git --version # Confirm Git is installed
git config --global user.name 'Pedro Amaral'
git config --global user.email 'your@email.com'
git config --global init.defaultBranch main
git config --global core.editor nano # Use nano for commit messages
```

2.4 The Daily Git Workflow

90% of your Git usage will follow this pattern:

THE CORE LOOP

```
git status          # See what has changed
git add .           # Stage all changes
git commit -m 'message' # Save a snapshot with a description
git push            # Upload to GitHub (if remote is set up)
```

✔ **Commit messages:** Write them like instructions, not descriptions. 'Add login form validation' is better than 'made some changes'. Future you will thank present you.

2.5 Starting a Project

INITIALIZING A NEW REPO

```
cd ~/vibe-projects
mkdir my-app && cd my-app
git init                # Turn this folder into a Git repo
touch .gitignore        # Create a file to list things Git should ignore
touch README.md
git add .
git commit -m 'Initial commit'
```

CLONING AN EXISTING REPO (FROM GITHUB)

```
git clone https://github.com/username/repo-name
cd repo-name
```

2.6 The .gitignore File

Some files should never be committed: API keys, node_modules (huge folder), build outputs, local config. The .gitignore file tells Git to ignore them.

```
# .gitignore — add these from day one
node_modules/
.env
.env.local
.DS_Store
dist/
build/
*.log
```

❏ **Pitfall:** Never commit API keys. If you accidentally push a key to GitHub, rotate it immediately. GitHub scanners and bots find exposed keys within minutes.

2.7 Branches

WORKING WITH BRANCHES

```
git branch                # List all branches (* = current)
git branch feature-login  # Create a new branch
git checkout feature-login # Switch to it
git checkout -b new-feature # Create AND switch in one command
git checkout main          # Go back to main
git merge feature-login    # Merge feature-login into current branch
git branch -d feature-login # Delete branch after merging
```

2.8 Undoing Things

This is the part people most need and least know. Git makes mistakes recoverable.

UNDOING CHANGES

```
git restore file.txt      # Discard uncommitted changes to a file
git restore .             # Discard ALL uncommitted changes
git reset HEAD~1          # Undo the last commit, keep the changes
git reset --hard HEAD~1   # Undo last commit AND discard the changes
git log --oneline         # See commit history with short IDs
git checkout abc1234      # Jump to any previous commit by its ID
```

❏ **Pitfall:** `git reset --hard` permanently discards changes. Only use it when you're certain you want to throw away your recent work.

2.9 Connecting to GitHub

PUSHING TO GITHUB

```
# On GitHub: create a new repo, then:
git remote add origin https://github.com/yourusername/repo-name.git
git push -u origin main    # First push: sets up tracking
git push                   # All subsequent pushes
git pull                   # Get latest changes from GitHub
```

2.10 Module 2 Exercises

9. In `~/vibe-projects`, create a folder called `git-practice` and run `git init` inside it.
10. Create three files, add some text to each, then commit them with a clear message.
11. Create a branch called `experiment`, make a change, then merge it back to `main`.
12. Create a `.gitignore` and add `node_modules/` and `.env` to it.
13. Run `git log --oneline`. You should see your commits listed.

14. Practice undoing: make a change, commit it, then run `git reset HEAD~1` to undo the commit (keep the changes).

MODULE 3

Node, npm, and Your Toolchain

Understanding the engine that powers your tools.

3.1 What Node.js Actually Is

Node.js is a JavaScript runtime — a program that can execute JavaScript code outside of a web browser. This is significant because JavaScript was originally a browser-only language. Node.js brought it to the server, the terminal, and build tools.

When you install tools like Gemini CLI, they are JavaScript programs that run on Node.js. npm (Node Package Manager) is the system that downloads, installs, and manages these programs and their dependencies.

You installed Node via nvm (Node Version Manager). This was deliberate: nvm lets you have multiple versions of Node on the same machine and switch between them, which matters because different projects often require different versions.

3.2 nvm Commands You'll Actually Use

NVM REFERENCE

```
nvm --version          # Confirm nvm is installed
nvm list               # See all installed Node versions
nvm list-remote --lts  # See available LTS versions online
nvm install --lts      # Install the latest Long Term Support version
nvm install 20         # Install a specific version
nvm use 20             # Switch to Node 20 in this session
nvm alias default 20   # Set Node 20 as the permanent default
node --version         # Confirm current Node version
which node             # Confirm it points to your nvm directory
```

❑ **LTS vs Current:** Always use LTS (Long Term Support) versions for real projects. They're stable and receive security patches for years. 'Current' versions get new features first but change faster.

3.3 npm: Package Management

Every modern JavaScript project has a package.json file. This is the project's manifest: it lists the project name, version, scripts, and — most importantly — dependencies (the external packages the project needs to work).

NPM ESSENTIALS

```
npm init -y           # Create a package.json with defaults
```

```
npm install express           # Install a package (adds to dependencies)
npm install -D jest           # Install a dev-only package
npm install -g gemini         # Install globally (available system-wide)
npm install                   # Install everything in package.json
npm uninstall express         # Remove a package
npm update                    # Update all packages
npm list                      # List installed packages
npm list -g --depth=0         # List global packages
npm run start                  # Run the 'start' script from package.json
npm run build                  # Run the 'build' script
```

3.4 Understanding package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "build": "next build"
  },
  "dependencies": {
    "express": "^4.18.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

The ^ before version numbers means 'compatible with this version'. npm will install the latest version that doesn't break compatibility.

3.5 node_modules and Why You Never Commit It

When you run npm install, it creates a node_modules folder containing all downloaded packages. This folder can contain thousands of files and be hundreds of megabytes. You never commit it to Git because:

- The package.json is all anyone needs to recreate it by running npm install
- It's machine-specific and would cause conflicts
- It makes your repo huge and slow

This is why node_modules/ is always in .gitignore.

3.6 Running Scripts

The scripts section of package.json is one of the most useful features in the Node ecosystem. It defines commands you can run with npm run.

COMMON SCRIPTS PATTERN

```
npm run dev           # Start development server (with hot reload)
npm start             # Start production server
npm run build         # Compile/bundle for production
npm test              # Run tests
```

❑ **Pro tip:** When you clone someone else's project, the first thing you do is npm install. The second thing is check their README for which npm run command starts the app.

3.7 Reading Error Messages

npm errors look scary but have a structure. Here's how to read them:

```
npm error code ENOENT
npm error errno -2
npm error syscall open
npm error path /Users/pedroamaral/project/package.json
```

Reading this:

```
code: ENOENT = No such file or directory
path: tells you WHAT file it couldn't find
Solution: you're probably in the wrong directory. Run pwd first.
```

Common error codes:

- ENOENT: File or directory not found. Check your path.
- EACCES: Permission denied. You're trying to write somewhere you shouldn't.
- ECONNREFUSED: Network connection refused. Check if a server is running.
- MODULE_NOT_FOUND: A required package isn't installed. Run npm install.

3.8 Module 3 Exercises

15. Run `nvm list` and note which version of Node is active.
16. Create a folder called `node-practice`. Run `npm init -y` inside it.
17. Open the `package.json` that was created. Add a script: `'hello': 'echo Hello from npm'`.
18. Run `npm run hello`.
19. Run `npm install chalk` (a package that adds colors to terminal output).
20. Look at `node_modules` and notice how many files were added for one package.
21. Add `node_modules/` to `.gitignore` and init a git repo. Confirm git status doesn't show `node_modules`.

MODULE 4

Gemini CLI: AI in the Terminal

Bringing conversational AI directly into your workflow.

4.1 What Gemini CLI Is

Gemini CLI is Google's command-line interface for interacting with Gemini AI models directly from your terminal. Unlike using Gemini through a browser, the CLI lets you integrate AI into scripts, pipe content into it, and use it as an agentic coding assistant inside your project directory.

The key distinction: Gemini CLI is context-aware of your filesystem. When you run it inside a project folder, it can read your files, understand your codebase, and make changes directly. This is fundamentally different from pasting code into a chat window.

4.2 Installation and Setup (You Already Did This)

CONFIRM IT'S WORKING

```
which gemini          # Should point to your nvm directory
gemini --version       # Confirm version
gemini                # Start an interactive session
```

If you need to configure your API key:

```
# Add this to your ~/.zshrc for persistence:
export GEMINI_API_KEY=your_api_key_here

# Then reload:
source ~/.zshrc
```

4.3 Interactive vs. Inline Usage

You can use Gemini CLI in two modes:

- Interactive: you type `gemini` and enter a session, like a chat. Good for back-and-forth.
- Inline: you pipe content directly to it in a single command. Good for automation.

INLINE USAGE PATTERNS

```
# Ask a direct question
gemini 'What does the fetch API do in JavaScript?'

# Pipe file content for analysis
cat index.js | gemini 'Explain what this code does'

# Ask for a fix
```

```
cat error.log | gemini 'What is causing this error and how do I fix it?'

# Generate code and save it
gemini 'Write a bash script that backs up my Desktop to ~/backups' > backup.sh
```

4.4 Using Gemini CLI Inside a Project

The real power is running Gemini CLI from inside your project directory. It can reference your actual files.

PROJECT-AWARE USAGE

```
cd ~/vibe-projects/my-app

# Ask about your specific codebase
cat src/App.js | gemini 'How can I improve this component?'

# Debug an error by piping the error output
npm run dev 2>&1 | gemini 'What is this error and how do I fix it?'

# Generate a specific file
gemini 'Write a .gitignore for a Next.js project' > .gitignore
```

4.5 Prompt Engineering for CLI

The quality of Gemini CLI's output depends almost entirely on how well you phrase your request. Here are patterns that consistently produce better results:

- Be specific about the technology stack. Say 'Next.js 14 with App Router' not just 'my project'.
- Ask for code examples AND explanations: 'Give me instructions and code examples'.
- Specify the format you want: 'Give me a bash script', 'Return only the JSON', 'Write this as a README'.
- Provide context about constraints: 'Without using any external libraries', 'Compatible with Node 20'.
- When debugging: paste the full error, not just part of it.

□ **The x___ pattern:** From the vibe coding rules: when asking for a feature, describe it as 'create [feature] and give me instructions AND code examples'. This forces the AI to show its reasoning, which helps you catch errors.

4.6 The Debug Loop

When something breaks, this is your loop:

22. Run the failing command and capture its full output.
23. Pipe that output to Gemini CLI with context about what you were trying to do.
24. Apply the suggested fix.
25. Run the command again.
26. If it still fails, include the new error AND the previous attempt in your next prompt.

THE DEBUG LOOP IN PRACTICE

```
# Step 1: Capture the error
npm run build 2>&1 | tee build-error.log

# Step 2: Send to Gemini with context
cat build-error.log | gemini 'I am trying to build a Next.js 14 app.
This is the full error. What is wrong and how do I fix it step by step?'

# Step 3: Apply fix, then try again
npm run build
```

❑ **Pitfall:** Don't just paste errors into Gemini without context. Always tell it what you were trying to do, what you expected, and what actually happened.

4.7 Module 4 Exercises

27. Run gemini and ask it to explain what nvm does. Exit with Ctrl+C or type /exit.
28. Create a JavaScript file with a simple function. Pipe it to gemini and ask for improvements.
29. Use gemini to generate a .gitignore for a Next.js project. Save it to a file.
30. Deliberately introduce an error in a JS file. Run node on it, capture the error, pipe to gemini for a fix.
31. Ask gemini to write a bash script that creates a new project folder with a .gitignore, README.md, and package.json already initialized.

MODULE 5

Antigravity IDE: Agentic Coding in Practice

From vibe to production — letting the agent drive.

5.1 The Agentic Coding Mindset

Traditional coding: you write every line. Assisted coding: AI suggests lines you accept or reject. Agentic coding: you describe what you want at a high level and the agent plans, writes, edits, and executes across multiple files autonomously.

The shift this requires is learning to think in terms of outcomes and constraints rather than implementation. Instead of thinking 'I need to write a function that validates email', you think 'the app should reject invalid email formats on the signup form with a specific error message'.

The agent handles implementation. Your job becomes specification, review, and direction.

5.2 Project Setup: The Right Foundation

Always start from a template, not from scratch. Templates encode best practices — folder structure, build config, authentication boilerplate — that would take hours to write yourself.

SETTING UP A NEW PROJECT

```
# Clone a battle-tested template
git clone https://github.com/ansh/template-2 my-project
cd my-project

# Install dependencies
npm install

# Immediately initialize your own git history
rm -rf .git
git init
git add .
git commit -m 'Initial commit from template'

# Start the dev server
npm run dev
```

□ **Template-2:** The template at github.com/ansh/template-2 comes pre-configured with Next.js, database integration, authentication, and AI features — saving you days of setup.

5.3 Working with Agent Mode

Agent mode (not normal chat mode) is where Antigravity becomes powerful. In agent mode, the AI can create, edit, and delete files based on your instructions — it acts on your project, not just responds to you.

Key rules for effective agent sessions:

- One task per session. Don't ask the agent to add a login system AND redesign the homepage in the same chat. Create a new chat for each distinct feature.
- Be specific about what done looks like. 'Add a user settings page' is vague. 'Add a user settings page at /settings that lets users change their display name and profile picture, using the existing auth system' is actionable.
- Keep sessions short. Long sessions accumulate context that can confuse the agent. Commit your work and start fresh for the next task.
- Review every change before committing. Read what the agent wrote. You don't need to understand every line, but you need to understand what it changed and why.

5.4 The Vibe Coding Workflow

This is the refined workflow that integrates the best practices from agentic coding:

32. Find designs or API references using Perplexity or Google AI Studio. Get context and code examples before starting.
33. Open a new agent chat in Antigravity. Paste your research. Write a clear specification of the feature.
34. Let the agent run. Watch what it does but don't interrupt unless it's clearly going wrong.
35. Test the result locally (npm run dev). Check it in your browser.
36. If it breaks: copy the full error and paste into a new agent chat with context.
37. When it works: commit immediately. git add . && git commit -m 'feat: add user settings page'
38. Move to the next feature in a fresh chat.

✔ **Commit granularly:** Commit after each working feature, not at the end of the day. If the agent breaks something in the next session, you can restore to the last good state with git reset.

5.5 Secrets and API Keys

This is non-negotiable. API keys and secrets live in .env files, never in code.

```
# .env.local (never commit this file)
NEXT_PUBLIC_APP_URL=http://localhost:3000
DATABASE_URL=postgresql://user:pass@localhost/mydb
GEMINI_API_KEY=AIza...
STRIPE_SECRET_KEY=sk_test...

# In your code, access them as:
process.env.GEMINI_API_KEY
```

```
process.env.DATABASE_URL
```

Your .gitignore must always contain:

```
.env  
.env.local  
.env.production
```

❑ **Pitfall:** If you commit a secret to GitHub — even briefly — assume it's compromised. Rotate the key immediately. GitHub's secret scanning and external bots index commits within minutes.

5.6 Restoring Previous States

The agent will sometimes make changes that seem correct but subtly break something elsewhere. The earlier you catch this, the easier the fix. This is why frequent commits are critical.

RESTORING AFTER A BAD AGENT RUN

```
# See recent commits  
git log --oneline  
  
# Undo last commit, keep changes staged  
git reset HEAD~1  
  
# Discard all uncommitted changes completely  
git restore .  
  
# Jump to a specific previous commit (creates a detached state)  
git checkout abc1234  
  
# Come back to main  
git checkout main
```

5.7 Voice-to-Text with Whispr Flow

Whispr Flow (or similar tools) lets you dictate prompts instead of typing them. For long, detailed specifications this dramatically improves your speed and the quality of your prompts — speaking naturally tends to produce more complete descriptions than typing.

Effective use: speak your specification as if explaining to a junior developer. Include the context, the expected behavior, and any constraints. Then paste the transcript into your agent chat, clean up any obvious transcription errors, and send.

5.8 Deploying Early

Deploy your app to Vercel or a similar platform early — ideally after the first working version, before it's 'finished'. This surfaces deployment-specific bugs that won't appear locally.

DEPLOYING TO VERCEL

```
# Install Vercel CLI
npm install -g vercel

# Login
vercel login

# Deploy (from your project directory)
vercel

# Deploy to production
vercel --prod
```

□ **Deploy early:** The first deployment is almost always the hardest. Surprises include missing environment variables, build-time errors that don't appear in dev, and platform-specific configs. Catching these early saves enormous pain later.

5.9 Module 5 Exercises

39. Clone the template-2 repo, run npm install, and start the dev server. Explore the folder structure.
40. Create a new Git repo for it with your own history (remove .git and reinit).
41. Create a .env.local with a fake API key. Confirm it's in .gitignore and doesn't appear in git status.
42. In Antigravity, open an agent chat. Ask it to add a simple 'About' page to the project. Review every file it changes.
43. Commit that change with a descriptive message.
44. Open a second, fresh agent chat. Ask it to add a footer to every page. Commit.
45. Practice the restore workflow: git log --oneline, then reset to before the footer was added.

MODULE 6

Fluency: Projects, Patterns, and Pitfalls

Putting it all together and leveling up.

6.1 What Fluency Looks Like

Fluency isn't memorizing commands. It's developing a mental model: knowing what's happening when something works, and knowing where to look when it doesn't. A fluent developer reads an error and immediately knows which layer it's from — the OS, Node, the framework, or their own code.

These are the markers of fluency in this stack:

- You know the difference between a permission error, a missing file error, and a network error — and how to fix each.
- You can set up a new project from scratch in under ten minutes.
- You commit regularly without thinking about it.
- You use the terminal for file management instead of Finder.
- You can read what an agent changed and evaluate whether it's correct.
- You know when to ask the AI and when to look at the docs.

6.2 The Mental Model of Layers

When something breaks, it's useful to know which layer it's from. Most problems are in one of these layers:

Your Code	← Logic errors, typos, wrong variable names
Your Config	← Wrong environment variables, missing .env values
Your Dependencies	← Wrong package version, missing npm install
Node/npm	← Version mismatch, global vs local conflict
The OS/Permissions	← File permission errors (EACCES), missing directories
The Network	← API keys wrong, rate limits, connectivity

When debugging, work top to bottom. Most bugs are in the top two layers.

6.3 A Cheatsheet: The Commands You'll Use Every Day

DAILY TERMINAL CHEATSHEET

```
# Navigation
pwd && ls -la           # Where am I and what's here
cd ~/vibe-projects/myapp # Jump to project

# Git (daily loop)
git status              # What changed?
```

```

git add . && git commit -m 'message'
git push

# Node/npm
node --version && which node # Am I on the right Node?
npm install # After cloning someone else's project
npm run dev # Start dev server

# Gemini CLI
cat error.log | gemini 'What is this error? How do I fix it?'
gemini 'Write a bash script that...' > script.sh

# Processes
Ctrl+C # Stop any running process
Ctrl+R # Search command history

```

6.4 Prompt Library — Save These

These are prompt patterns that consistently produce excellent results. Copy them and adapt:

```

# Feature request
I am building a [type] app using [stack]. I want to add [feature].
The feature should [specific behavior]. It should use [existing system].
Give me step-by-step instructions AND code examples.

# Debugging
I am working on a [stack] project. I ran [command] and got this error:
[paste full error]
I was trying to [goal]. What is causing this and how do I fix it?

# Code review
Review this [language] code for: correctness, security, performance,
and readability. Point out any issues and suggest improvements.
[paste code]

# Architecture advice
I am building [description]. My constraints are [constraints].
What is the simplest architecture that solves this? Avoid over-engineering.

```

6.5 Common Pitfalls and How to Avoid Them

❑ **Pitfall:** Working without Git. Even for 'quick tests'. The one time you need it and don't have it will cost you hours.

❑ **Pitfall:** Using `sudo npm install`. You fixed this with `nvm`. If you ever see a permissions error again, never reach for `sudo` — instead, check which node is active and whether `nvm` is properly configured.

❑ **Pitfall:** Running agent sessions for multiple features at once. Long context confuses agents. Short, focused sessions with commits between them produce far better results.

❑ **Pitfall:** Committing without reading agent changes. The agent can introduce subtle bugs. Make it a habit to run `git diff` before `git add`.

❑ **Pitfall:** Ignoring deprecation warnings. The glob security warning you saw during Gemini CLI install was real. Warnings compound over time. Stay updated.

6.6 Keeping Your Prompts — The Prompt Ledger

As you develop prompts that reliably produce great results, write them down. Keep a file called `PROMPTS.md` in your home directory or a Notion page. Note:

- What the prompt was
- What project or context it was for
- What it produced
- Any refinements that improved it

After a few months, this becomes one of your most valuable assets. Prompt engineering is a skill that compounds — small improvements in how you ask questions produce dramatically better outputs over time.

6.7 What to Learn Next

Once you're comfortable with this stack, here's the natural progression:

- SQL basics: understanding how databases work will make you dramatically more effective. Start with SQLite, then move to PostgreSQL.
- HTTP and APIs: learn how web requests work — what a GET and POST are, what headers are, what a 200 vs 400 vs 500 response means. This unlocks debugging anything network-related.
- TypeScript: JavaScript with types. The `template-2` repo uses it. It catches errors before they reach production.
- Docker: learn to containerize your apps. Eliminates 'works on my machine' problems permanently.
- Understanding Next.js: learn the difference between server-side rendering, static generation, and client-side rendering. This is fundamental to building fast web apps.

6.8 Final Capstone Project

Build this to prove your fluency:

46. Clone template-2 into a new project called capstone-app.
47. Set up your .env.local with a real Gemini API key.
48. Using Antigravity in agent mode (one feature per session), build: a home page, an about page, a contact form that validates input, and a results page that shows a Gemini-generated response to a user's question.
49. Commit after each working feature. Your git log should have at least 6 commits.
50. Add a proper README.md explaining what the app does.
51. Deploy to Vercel.
52. Share the live URL.

□ **You're fluent:** If you can complete the capstone project without getting stuck for more than 30 minutes on any single problem, you have reached the fluent tier. Everything beyond this is specialization.

Quick Reference

Terminal Commands

NAVIGATION

```
pwd
ls -la
cd [dir]
cd ..
cd ~
cd -
```

FILES

```
mkdir [name]
touch [file]
cp [src] [dest]
mv [src] [dest]
rm [file]
rm -rf [dir]
```

READING

```
cat [file]
less [file]
head -N [file]
tail -N [file]
tail -f [file]
```

PROCESS

```
Ctrl+C # stop
Ctrl+Z # pause
Ctrl+R # history search
Ctrl+L # clear
```

Git Commands

SETUP

```
git config --global user.name 'Name'
git config --global user.email 'email'
```

DAILY

```
git status
```

```
git add .
git commit -m 'msg'
git push
git pull
```

BRANCHES

```
git branch
git checkout -b [name]
git checkout main
git merge [branch]
```

UNDO

```
git restore .
git reset HEAD~1
git reset --hard HEAD~1
git log --oneline
```

npm / nvm Commands

NVM

```
nvm list
nvm install --lts
nvm use [version]
nvm alias default [version]
```

NPM

```
npm install
npm install [pkg]
npm install -g [pkg]
npm run [script]
npm list -g --depth=0
```

Gemini CLI

USAGE

```
gemini                                # Interactive session
gemini 'question'                     # Direct question
cat file | gemini 'prompt'            # Pipe content
gemini 'prompt' > file.txt            # Save output
npm run dev 2>&1 | gemini 'fix this error'
```

Built for Pedro Amaral · February 2026
MacOS · Terminal · nvm · Git · Gemini CLI · Antigravity IDE